사전에 정의된 체인

테디노트의 랭체인을 활용한 RAG 비법노트 심화편 | CHAPTER 03

이경록(테디노트) 저 · PART 01 고급 RAG

오늘 다룰 내용

📂 문서 요약 체인 (4가지)

1. **Stuff** 요약 — 한 번에 전부
2. **Map-Reduce** 요약 — 나눠서 합치기
3. **Map-Refine** 요약 — 순차 정제
4. **Chain of Density** — 밀도 높이기

🔧 고급 요약 & DB 쿼리

5. **Clustering-Map-Refine** — 클러스터링 + 요약
6. **SQL 쿼리 생성기** — 자연어 → SQL

핵심 키워드: `create_stuff_documents_chain` · `@chain` · `LangChain Hub` · `CoD` · `KMeans` · `create_sql_query_chain`

문서 요약 전략 — 전체 비교

전략	방식	장점	단점	적합 케이스
Stuff	전체→LLM	단순, 빠름	토큰 제한	~100페이지
Map-Reduce	분할→요약→합산	대규모 가능	순서 섞임	대규모 문서
Map-Refine	분할→순차 정제	맥락 유지	느림, 비쌈	흐름 중요시
CoD	반복 밀도 증가	고밀도 요약	비용↑	핵심 추출
Cluster	군집→대표→요약	초대규모	구현 복잡	100p+
SQL	NL→SQL→실행	DB 조회 자동화	정확도	구조화 데이터

01 Stuff 요약

문서 전체를 한 번에 LLM에 넣어 요약하는 가장 단순한 방식

Stuff 요약 — 개념

동작 원리

📄 문서 전체 → 🤖 LLM (1회 호출) → 📝 요약 결과

특징

- **가장 단순한 구현** — 코드 몇 줄로 완성
- `create_stuff_documents_chain(llm, prompt)` 사용
- LLM의 **컨텍스트 윈도우**가 문서를 수용해야 함

제약 사항

- 128K 모델 기준 → A4 **50~100페이지**
- 대규모 문서에는 **부적합**

Stuff 요약 — 구현 코드

1 문서 로드 & 프롬프트 다운로드

```
from langchain_community.document_loaders import TextLoader
from langchain import hub

loader = TextLoader("data/news.txt")
docs = loader.load()  # 1,708자 분량의 뉴스 기사

# LangChain Hub에서 한국어 요약 프롬프트 가져오기
prompt = hub.pull("teddynote/summary-stuff-documents-korean")
```

2 체인 생성 & 실행

```
from langchain_openai import ChatOpenAI
from langchain.chains.combine_documents import create_stuff_documents_chain

llm = ChatOpenAI(model_name="gpt-4o-mini", temperature=0, streaming=True)
stuff_chain = create_stuff_documents_chain(llm, prompt)
answer = stuff_chain.invoke({"context": docs})
```

- 커스텀 프롬프트 사용 시 `PromptTemplate.from_template()`에 `{context}` **변수 필수**

02 Map-Reduce 요약

문서를 청크로 나눠 각각 요약 → 하나로 합산하는 방식

Map-Reduce — 개념

동작 다이어그램

📄 청크①

📄 청크②

📄 청크③

→

요약1

요약2

요약3

→

📝 최종 요약

Map (병렬) → Reduce

핵심 포인트

- **토큰 제한에 구애받지 않음**
- Map: 청크별 핵심 추출 (`batch()` 병렬)
- Reduce: 요약본들을 최종 통합
- `map_prompt` + `reduce_prompt` 별도 프롬프트

⚠️ 주의

- 합치는 과정에서 **순서가 섞일 수 있음**
- 흐름보다 **빠른 처리**가 중요할 때 적합

Map-Reduce — 구현 코드

Map 단계: 각 청크를 요약

```
from langchain import hub
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(temperature=0, model_name="gpt-4o-mini")
map_prompt = hub.pull("teddynote/map-prompt")       # 핵심 요점 추출 프롬프트
map_chain = map_prompt | llm | StrOutputParser()
doc_summaries = map_chain.batch(docs)               # 병렬로 모든 청크 요약
```

Reduce 단계: 요약본들을 하나로 통합

```
reduce_prompt = hub.pull("teddynote/reduce-prompt")  # 통합 요약 프롬프트
reduce_chain = reduce_prompt | llm | StrOutputParser()

answer = reduce_chain.stream(
    {"doc_summaries": "\n".join(doc_summaries), "language": "Korean"}
)
```

03 Map-Refine 요약

Map 단계는 동일, Reduce 대신 순차적으로 정제하는 방식

Map-Refine — 개념 & 비교

동작 다이어그램

📄 청크①

📄 청크②

📄 청크③

📄 청크④

→

요약1

요약1+2

요약1+2+3

최종 요약

Map → 순차 Refine (이전 요약에 누적)

Map-Reduce와 차이점

항목	Map-Reduce	Map-Refine
Reduce 방식	한 번에 합산	순차적 정제
순서 보존	❌ 섞일 수 있음	✅ 유지됨
요약 밀도	보통	**높음**
비용/시간	저렴/빠름	비쌈/느림
적합 케이스	빠른 처리	맥락 중요

Map-Refine — 구현 코드

Refine 프롬프트 (`teddynote/refine-prompt`)

```
refine_prompt = hub.pull("teddynote/refine-prompt")
# "이전 요약(previous_summary)과 현재 요약(current_summary)을 합쳐
#  {language}로 최종 요약을 정제하세요"
```

Refine 루프 — 순차적으로 요약 정제

```
refine_chain = refine_prompt | refine_llm | StrOutputParser()

previous_summary = doc_summaries[0]        # 첫 번째 요약으로 시작
for current_summary in doc_summaries[1:]:  # 나머지를 순차적으로 정제
    previous_summary = refine_chain.invoke({
        "previous_summary": previous_summary,
        "current_summary": current_summary,
        "language": "Korean",
    })
```

04 Chain of Density (CoD) 요약

반복적으로 엔티티를 추가하며 밀도 높은 요약을 만드는 기법

Chain of Density — 개념

반복 밀도 증가 프로세스

1️⃣ 초기 요약 (max_words 이내, entity_range개 엔티티 포함)

⬇ **누락 엔티티 탐색**

2️⃣ 새 엔티티 추가 + 같은 길이로 더 밀도 높은 요약 생성

⬇ **반복 (iterations 횟수)**

🎯 최종 고밀도 요약

입력 매개변수

매개변수	기본값	설명
`content_category`	Article	콘텐츠 종류
`content`	-	요약할 대상
`entity_range`	1~3	추가 엔티티 수
`max_words`	80	요약 최대 단어
`iterations`	5	반복 횟수

Descriptive Entity 조건

- **관련**(Relevant) · **구체적**(Specific, 5단어 이하)
- **새로움**(Novel) · **충실**(Faithful) · 원문 어디서나 찾을 수 있음

Chain of Density — 구현 코드

체인 구성

```python
from langchain import hub
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import JsonOutputParser

cod_chain_inputs = {
    "content": lambda d: d.get("content"),
    "content_category": lambda d: d.get("content_category", "Article"),
    "entity_range": lambda d: d.get("entity_range", "1-3"),
    "max_words": lambda d: int(d.get("max_words", 80)),
    "iterations": lambda d: int(d.get("iterations", 5)),
}
cod_prompt = hub.pull("teddynote/chain-of-density-prompt")

cod_chain = (
    cod_chain_inputs
    | cod_prompt
    | ChatOpenAI(temperature=0, model="gpt-4o-mini")
    | JsonOutputParser()
)

# → 최종 요약 추출
cod_final_summary_chain = cod_chain | (
    lambda output: output[-1].get("denser_summary", "키 없음")
)
```

Chain of Density — 반복 결과 예시

반복할수록 엔티티가 추가되며 밀도 증가

반복	추가된 엔티티	요약 예시
1/5	G7, AI 국제 행동강령	G7은 히로시마 AI 프로세스를 통해 AI 기업을 위한 국제 행동강령에 합의
2/5	위험 식별, 첨단 AI 시스템	+첨단 AI 시스템의 위험 식별, 평가 및 완화
3/5	물리보안, 사이버보안	+물리보안, 사이버보안, 콘텐츠 인증
4/5	사회적 위험, 기후 위기	+사회적 위험 완화, 기후 위기 대응
5/5	AI 거버넌스, 신뢰 콘텐츠	+AI 거버넌스, 위험 관리 정책

💡 **핵심:** 같은 길이(max_words) 안에서 점점 더 많은 핵심 정보를 담아내는 것이 CoD의 강점

05 Clustering-Map-Refine 요약

초대규모 문서를 클러스터링하여 대표 문서만 요약하는 전략

Clustering-Map-Refine — 전체 파이프라인

📄 전체 문서 (28K자) → ✂️ 텍스트 분할 (79개 청크) → 🧮 임베딩 벡터화 → 📊 KMeans 클러스터링 → 🎯 대표문서 10개 선정 → 📝 Map-Refine 요약

왜 클러스터링인가?

- 79개 청크를 **모두 요약하면 비용과 시간이 과다**
- KMeans로 **10개 클러스터**로 그룹화 → 각 클러스터의 **중심점에 가장 가까운 문서** 1개 선정
- 10개 대표 문서만으로 **전체 문서의 핵심 주제를 커버**
- 대표 문서를 **오름차순 정렬** 후 Map-Refine으로 요약 → 문서 흐름 유지

Clustering — 임베딩 & 클러스터링

1 문서 분할 & 임베딩

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings

text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
split_docs = text_splitter.split_text(texts)  # 79개 청크

embeddings = OpenAIEmbeddings()
vectors = embeddings.embed_documents(split_docs)
```

2 KMeans 클러스터링 & 대표 문서 선정

```
from sklearn.cluster import KMeans
import numpy as np

kmeans = KMeans(n_clusters=10, random_state=123).fit(vectors)

closest_indices = []
for i in range(10):  # 각 클러스터의 중심점에 가장 가까운 문서 인덱스
    distances = np.linalg.norm(vectors - kmeans.cluster_centers_[i], axis=1)
    closest_indices.append(np.argmin(distances))

selected_indices = sorted(closest_indices)   # 오름차순 정렬로 문서 순서 유지
```

Clustering — t-SNE 시각화

시각화 코드

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import seaborn as sns

tsne = TSNE(n_components=2,
            random_state=42)
reduced = tsne.fit_transform(
    np.array(vectors)
)

sns.scatterplot(
    x=reduced[:, 0],
    y=reduced[:, 1],
    hue=kmeans.labels_,
    palette="deep", s=100
)
plt.title("Clustered Embeddings")
```

시각화 결과의 의미

- 가까운 공간에 있는 문서들 → **같은 클러스터**
- 각 클러스터의 **중심(centroid)** 에 가장 가까운 문서를 대표로 선정
- 10개 클러스터 → **10개 대표 문서**
- 주제의 다양성이 높으면 `n_clusters` 증가 가능

💡 **핵심:** 79개 → 10개로 줄여도 전체 주제 커버 가능. 비용과 품질의 균형점

06 SQL 쿼리 생성기

자연어 질문을 SQL로 변환하고 실행 결과를 반환하는 체인

SQL 쿼리 생성기 — 개념

동작 흐름

💬 자연어 질문 → create_sql_query_chain (NL→SQL) → QuerySQLDataBaseTool (SQL 실행) → 📊 결과 반환

구성 요소

- `SQLDatabase.from_uri()` — DB 연결 (SQLite, PostgreSQL, MySQL 등)
- `create_sql_query_chain()` — NL→SQL 변환 · `QuerySQLDataBaseTool()` — SQL 실행
- 두 체인을 **파이프(|)** 로 연결 · 체인이 **테이블 정보를 자동 조회**하여 프롬프트에 포함

SQL 쿼리 생성기 — 구현 코드

기본 체인 구성

```
from langchain_openai import ChatOpenAI
from langchain.chains import create_sql_query_chain
from langchain_community.utilities import SQLDatabase
from langchain_community.tools.sql_database.tool import QuerySQLDataBaseTool

db = SQLDatabase.from_uri("sqlite:///data/finance.db")
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

write_query = create_sql_query_chain(llm, db)     # NL → SQL
execute_query = QuerySQLDataBaseTool(db=db)        # SQL 실행
chain = write_query | execute_query                # 파이프라인
chain.invoke({"question": "테디의 이메일을 조회하세요"})
```

자연어 답변까지 (LCEL 활용)

```
from langchain_core.runnables import RunnablePassthrough
from operator import itemgetter

chain = (
    RunnablePassthrough.assign(query=write_query)
    .assign(result=itemgetter("query") | execute_query)
    | answer_prompt | llm | StrOutputParser()
)
```

SQL 에이전트 방식

`create_sql_agent` — 쿼리 생성부터 답변까지 자동화

```
from langchain_community.agent_toolkits import create_sql_agent

agent_executor = create_sql_agent(
    llm, db=db, agent_type="openai-tools", verbose=True
)
agent_executor.invoke(
    {"input": "테디와 셜리의 transaction의 합계를 구하고 비교하세요"}
)
```

에이전트 자동 실행 흐름

1. `sql_db_list_tables` → 테이블 목록 조회
2. `sql_db_schema` → 테이블 구조 파악
3. `sql_db_query` → SQL 실행 (필요시 여러 번)
4. 결과를 자연어로 정리하여 응답

⚠️ **참고:** `create_sql_agent`는 LangChain 레거시 API. 최신 버전에서는 `langgraph` 기반 에이전트 권장

전략 선택 가이드

짧은 문서 (< 100p) → **Stuff** 요약
가장 빠르고 단순

긴 문서 · 빠른 처리 → **Map-Reduce** 요약
병렬 처리로 속도 확보

긴 문서 · 맥락 중요 → **Map-Refine** 요약
순차 정제로 품질 확보

핵심 엔티티 추출 → **Chain of Density**
밀도 높은 요약 필요 시

초대규모 문서 (100p+) → **Clustering-Map-Refine**
클러스터링으로 대표 추출

구조화 데이터 질의 → **SQL 쿼리 생성기**
DB 연동 자연어 QA

LangChain Hub 프롬프트 정리

프롬프트 ID	용도	섹션
`teddynote/summary-stuff-documents-korean`	한국어 Stuff 요약	01
`teddynote/map-prompt`	Map 단계 핵심 추출	02, 03
`teddynote/reduce-prompt`	Reduce 통합 요약	02
`teddynote/map-summary-prompt`	Map 요약 (언어 지정)	03
`teddynote/refine-prompt`	Refine 순차 정제	03
`teddynote/chain-of-density-prompt`	CoD 밀도 증가	04

💡 **LangChain Hub** — 검증된 프롬프트를 `hub.pull("owner/name")` 로 즉시 사용.
최신: `langsmith` SDK의 `client.pull_prompt()` 도 지원. 커스텀은 `PromptTemplate` 로 작성

핵심 코드 패턴 정리

이 챕터에서 반복 등장하는 LCEL 패턴

```
# 1. 파이프 연산자로 체인 구성
chain = prompt | llm | StrOutputParser()

# 2. batch()로 병렬 처리
results = chain.batch(docs)

# 3. @chain 데코레이터로 함수를 체인으로 변환
from langchain_core.runnables import chain
@chain
def my_chain(inputs):
    # 복잡한 로직 ...
    return result

# 4. RunnablePassthrough.assign()으로 중간 결과 저장
chain = (
    RunnablePassthrough.assign(query=write_query)
    .assign(result=itemgetter("query") | execute_query)
    | answer
)
```

⚠️ 최신 API 변경사항 (2025)

LangChain v1.0 대규모 구조 변경

항목	기존 (책 기준)	최신 (2025)
`create_stuff_documents_chain`	`langchain.chains`	`langchain-classic` 패키지로 이동
`create_sql_query_chain`	`langchain.chains`	`langchain-classic` 패키지로 이동
`hub.pull()`	`from langchain import hub`	`langsmith` SDK 권장
`SimpleJsonOutputParser`	`langchain_core`	`JsonOutputParser` 의 별칭 (동일 클래스)
`create_sql_agent`	`langchain-community`	`langgraph` 기반 에이전트 권장

변경되지 않은 항목

- `@chain`, `RunnablePassthrough`, `StrOutputParser` — **langchain-core 유지**
- `QuerySQLDataBaseTool` — **langchain-community 유지** · CoD 기법 — **유효** · teddynote/* 프롬프트 — **접근 가능**
- 💡 책의 코드는 LangChain 0.2~0.3 기준 → `pip install langchain-classic` 으로 호환성 유지 가능

Q & A

감사합니다